

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14 COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: Total Marks:100

Annexure A Debugging

Code of Conduct:

I Vishal B(192524485) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Vishal B

CO3 - ASSESSMENT

TOOL - 2

Name: B. vishal

Reg No: 192524485

Course : compilation design
code :

course : CSA-1408
Name :

1. Error:

a. The semantic rule associated with the production $E \rightarrow E_1 + T$ is incorrectly defined as $\{E.val = T.val\}$

a. Synthesized attributes are computed from the attribute values of a node's children.

a. In the incorrect rule $\{E.val = T.val\}$, the semantic action completely ignores the values synthesized by E_1 ($E_1.val$).

a. During evaluation of $3+5$, E_1 evaluates to 3 and T evaluates to 5, the rule discards 3 and assigns $E.val = 5$

a. Corrected semantic Rule:

$$E \rightarrow E_1 + T \quad \{E.val = E_1.val + T.val\}$$

Correct value computation

$$1 \rightarrow T \rightarrow \text{num} \Rightarrow T.val = 3$$

$$2 \rightarrow E_1 \rightarrow T \Rightarrow E_1.val = T.val = 3$$

$$3 \rightarrow T \rightarrow \text{num} \Rightarrow T.val = 5$$

$$4 \rightarrow E \rightarrow E_1 + T \Rightarrow E.val = 3 + 5 = 8$$

Q. Error:

• The syntax tree constructed places the addition operator $[+]$ below the multiplication operator $[*]$.

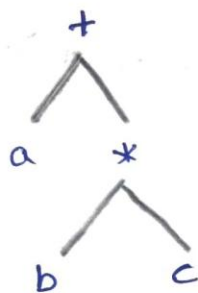
• In an Abstract Syntax Tree (AST), sub trees located at lower depth levels are evaluated first.

• Consulting, constructing $(+)$ lower than $(*)$ assigns higher precedence to addition.

• Consequently the compiler evaluates $(a+b)$ first generating $(t_1 = a+b)$ and then multiplies by $(t_2 = t_1 * c)$.

• Standard programming language grammar dictates that multiplication $(*)$ has higher precedence than addition $(+)$.

• Corrected syntax tree:



• Correct Intermediate Representation:

$t_1 = b * c;$

$t_2 = a + t_1;$

3. Error:

* The compiler incorrectly computes the root attribute as

$E.val = 25$ for the expression $3 + 4 * 5$.

* Parse tree of Attribute tree:

$$1) F \rightarrow \text{num}(3) \Rightarrow F.val = 3$$

$$T_1 \rightarrow F \Rightarrow T_1.val = 3$$

$$E_1 \Rightarrow T_1 \Rightarrow E_1.val = 3$$

$$2) F \rightarrow \text{num}(4) \Rightarrow F.val = 4;$$

$$T_2 \rightarrow F \Rightarrow T_2.val = 4$$

$$3) F \rightarrow \text{num}(5) \Rightarrow F.val = 5;$$

$$4) T_3 \rightarrow T_2 * F \Rightarrow T_3.val = 4 * 5 = 20$$

$$5) E_2 \rightarrow E_1 + T_3 \Rightarrow E_2.val = 3 + 20 = 23$$

* Root cause:

* The erroneous result occurred because the semantic evaluation performed a left $\rightarrow$ to right evaluation ($3 + 4 * 5 \neq 35$) disregarding the operator hierarchy enforced by the context free grammar

$$[E \rightarrow E + T (T, T \rightarrow T * F | F)]$$

* Correct result:

$$E.val = 23$$

4. Error:

* The compiler raised a false positive type error: incompatible operands for the expression $x = a + b * c$.

* operand types:

a: int
b: float
c: int
d: float

* Subexpression 1 $(b * c)$:

* operands are float and int. Type hierarchy requires evidencing c from int to float via implicit coercion [int to float (c)].

Resulting type: float

* Subexpression 2 $(a + (b * c))$:

* operands are a (int) and the result of $(b * c)$ (float). a must be implicitly converted to float (int to float(a)).

Resulting type: float

* Assignment $(x = \text{expression})$: Target variable x is float

and expression evaluates to float. Types match without narrowing loss.

* The statement is type valid the error occurred because the type checker lacked implicit type promotion coercion and logic.

5. Error:

* The type checker incorrectly flagged both assignment as valid without inserting mandatory implicit conversion instruction nodes into the AST.

* Statement 1 ($x = a + b$ where $a: \text{int}, b: \text{float}, x: \text{int}$);

1. Addition ($a + b$):

* operand $a(\text{int})$ must be promoted to float (int to float) (x) before addition. Addition yields float.

2. Assignment ($x = \text{result}$):

* Assigning a float result to an int target (x) requires explicit or implicit occurring coercion (float to int) to prevent unstable type assignment.

* Statement 2 ($b = a + x$ where $a: \text{int}, x: \text{int}, b: \text{float}$):

1-) Addition ($a + x$):

* Both operands are int, yielding an int result.

2-) Assignment ($b = \text{result}$):

* Assigning an int result to a float (b) requires implicit

underlying conversion (int to float).

$\therefore$ The type checker must transform expression into an

intermediate representation containing explicit conversion nodes.